

Java Script

Syllabus

- ✓ Variables (var, let, const)
- ✓ Functions
- ✓ Prototype
- ✓ Json
- ✓ Promises
- ✓ AJAX Calls
- ✓ CallBacks
- ✓ Clousers
- ✓ IIFE
- ✓ Data Structures
- ✓ Event Bubbling & Event Capturing
- ✓ Regular Expressions
- ✓ Generators & Iterators
- ✓ Arrays Manipulations



Functions

=>Default parameters in functions:

- while defining the functions, few parameters are initialized to default values, That concept is called default parameters.
- Introduced in ES6 version

Examples:

=>

```
function fun_one(arg1 = "ReactJs", arg2 = "AngularJs", arg3 =  
"MongoDB")
```

```
{  
  console.log(arg1, arg2, arg3);  
}
```

```
fun_one();
```

```
//ReactJs AngularJs MongoDB
```

```
fun_one("Java");
```

```
//Java AngularJs MongoDB -only arg1 will be changed. other  
remain default values
```

```
fun_one(undefined, undefined, undefined);
```

```
//ReactJs,AngularJs,MongoDB,undef  
cannot change the default values
```

```
fun_one(null, null, null);
```

```
//null null null, null will override default values
```



=>: (Combination of Regular parameters, Default parameters)

```
function fun_one(arg1, arg2 = "AngularJs", arg3) {
  console.log(arg1, arg2, arg3);
}
fun_one();
//undefined AngularJs undefined --- "Variable Hoisting".

fun_one("Java");
// Java AngularJs undefined - only ar2 will be changed. other
remain default values

fun_one(undefined, undefined, undefined);
//undefined AngularJs undefined, undef cannot change the default
values

fun_one(null, null, null);
//null null null, null will override default values
```

Variable hoisting

- If we are calling regular parameter without defining value, then undefined will be assigned to that variable. This is called Variable hoisting.

=>: (combination of Regular parameters, Default Parameters , Rest Paramets)

```
function fun_one(arg1, arg2 = "AngularJs", ...arg3)
{
  console.log(arg1, arg2, arg3);
} fun_one(); //undefined AngularJs [ ]
```



```
fun_one("Java", undefined, "Html", "css");
// Java AngularJs [ 'Html', 'css' ] - only arg2 will be changed.
other remain default values

fun_one(undefined, undefined, undefined);
//undefined                undefined                [undefined],
undef cannot change the default values

fun_one(null, null, null);
//null null null, null will override default values
```

Arrow Functions:

- Introduced in ES6, represent using => symbol
- Also called as Fat Arrow function / Anonymous function (function without name is called as anonymous function)
- Handle events (click, touch , mouse over etc...) and bind them (events) □ Application performance is better compared to regular function (Theoretically)
- Object creations for functions is NOT possible in Arrow Functions i.e
 - Representation of Data is NOT possible
- If we want to read response from server - Arrow functions are suggestible functions Syntax:

```
Let variableName = ( arguments ... ) => {...Business Logic...}
```

Examples

=>

```
let var1 = () => {
  console.log("Hello World");
}
```



```
}  
console.log(var1);  
//[Function:var1]  
console.log(var1());  
// Hello World
```

=>Example in usage of arrow functions in html
(filename.html):

```
<!DOCTYPE html>  
<html>  
<head>  
    <meta charset='utf-8'>  
    <meta http-equiv='X-UA-Compatible' content='IE=edge'>  
    <title>Arrow Functions</title>  
    <script>  
let my_fun = () =>  
{  
return "Hello World....!"  
}  
console.log(my_fun);  
// will give function definition --NOT Value  
console.log(my_fun());  
// will give its value ( Business Logic )  
    </script>  
</head>  
<body>  
</body>  
</html>
```



o/p:

```
() =>
{
Return"Hello World....!"
}
```

Hello World....!

Nested Arrow Functions:

```
<script>
let my_fun = () => {
return () => {
    return "Hello World....!"
}
}
console.log(my_fun);
//      outer      function      definition
console.log(my_fun());
//      inner      function      definition
console.log(my_fun() ());
// inner function result...!
</script>
```

Arrow Functions with arguments:

```
<script>
let my_fun = (arg1, arg2) =>
{
console.log(arg1, arg2);
}
```



```
my_fun("ReactJs", "AngularJs");  
</script>
```

o/p:

ReactJs AngularJs

Arrow functions as arguments

```
    <script>  
let  asyncFun  =  (successResponse,  errorResponse)  =>  {  
  console.log(successResponse);  
  //           Returns           function           definition  
  console.log(successResponse());  
  // Return function output  
  console.log(errorResponse);  
  //           Returns           function           definition  
  console.log(errorResponse());  
  // Returns function output  }  
  
asyncFun( () => {  
    return "Success Message";  
    }, // Argument 1  
    ()=>{  
    return "Error Message";  
    } // Argument 2  
  
    ) // Function ends  
    </script>
```

Push arrow function definition to array upto 5 elements

```
<script>
```



```
function myFun() {  
  let myarray = [];  
    for (let i = 0; i < 5; i++)  
    {  
      myarray.push(  
        () => { return "Welcome " + i + "th time"; }  
      )  
    }  
    for (let i = 0; i < 5; i++) {  
      console.log(myarray[i]());  
    }  
  }  
  myFun();  
</script>
```

o/p:

```
Welcome 0th time  
Welcome 1th time  
Welcome 2th time  
Welcome 3th time  
Welcome 4th time
```

setTimeout()

- Execute a function after a particular time.....

Syntax : `setTimeout(functionName(), time_in_mSec's)`

Example 1:

```
<script>
```

```
  myFun = () => {
```




```
        console.log("Hello..");
    }
    setTimeout(myFun, 5000);
</script>
```

Example 2: arrow function definition as parameter for setTimeout

```
<script>
setTimeout(()    =>    {
    console.log("Hello..");
    }, 5000);
</script>
```

Example 3: Combination of regular statements with setTimeouts

```
<script>
console.log("Hello 1");
setTimeout(() => {console.log("Hello 2");},
5000);
console.log("Hello 3");
</script>
```

o/p:

Hello 1

Hello 3

(After 5 seconds) Hello 2

//This mechanism is called "Event loop mechanism". Hello2 will be executed in parallel. This is secondary thread.



Example 3: Combination of regular statements with setTimeouts with 0 priority

```
<script>
console.log("Hello 1");
setTimeout(() =>{console.log("Hello 2");},
0);
console.log("Hello 3");
</script>
```

o/p:

Hello 1

Hello 3

Hello 2 executed by secondary thread.

setInterval()

- Execute a function for every interval of time.

Example 1:

```
let myFun = () => {
console.log("Helloworld..")
}
setInterval(myFun, 1000);
```



o/p: prints Hello world for every one second.

Example 2:

```
let myFun = () => {  
    console.log(new Date().toLocaleTimeString());  
}  
setInterval(myFun,  
1000);
```

o/p: Displays current time for every 1 second.

Assignment: execute setInterval function only 5 times...

Solution:

```
let count = 0; let timerId = 0; let myFun  
= () => { console.log(new  
Date().toLocaleTimeString());  
count++; if (count > 5) {  
clearInterval(timerId);  
}  
}  
timerId = setInterval(myFun, 1000, 3);
```

o/p: prints time only 5 times.



=>Constructor Functions:

- Used to create objects, using "new" keyword
- "this" keyword is used to refer current class content
- Object will be created in heap memory

Example:

```
function myConstructorFunction() {  
  this.sub_one      =      "ReactJs";  
  this.sub_two      =      "NodeJs";  
  this.sub_three = "AngularJs"  
}  
  
let myObj = new myConstructorFunction();  
console.log(myObj.sub_one, myObj.sub_two, myObj.sub_three);
```

o/p: ReactJs NodeJs AngularJs

=>Parameterized Constructor

```
function myConstructorFunction(sub_one, sub_two,  
sub_three) {  
  this.sub_one = sub_one;  
  this.sub_two = sub_two;  
  this.sub_three = sub_three;  
}  
  
let  myObj1  =  new  myConstructorFunction("ReactJs",  
"AngularJs", "NodeJs");
```



```
let myObj2 = new myConstructorFunction("Java", "Adv.Java",  
"Spring");
```

```
console.log(myObj1.sub_one,myObj1.sub_two,  
myObj1.sub_three);
```

```
console.log(myObj2.sub_one,myObj2.sub_two,  
myObj2.sub_three);
```

o/p:

ReactJs AngularJs NodeJs

Java Adv.Java Spring

Define Functions inside Constructor Functions:

```
function  
myConstructorFunction()      {  
  this.MEAN   =   "MEAN   Stack";  
  this.MERN   =   "MERN   Stack";  
  this.MAVN   =   "MAVN   Stack";  
  this.meanFunction=  
  function() {  
    return this.MEAN;  
  };  
  this.mernFunction=  
  function() {  
    return this.MERN;  
  };  
  this.mavnFunction =
```



```
function() {  
  return this.MAVN;  
};  
}  
  
let myObj = new  
myConstructorFunction();  
console.log(myObj.meanFunction(), myObj.mernFunction(),  
myObj.mavnFunction());  
o/p: MEAN Stack MERN Stack MAVN Stack
```

Function passed to another function as object / argument:

```
function funOne(arg1) {  
  this.test = arg1;  
}; function funtwo() { this.test =  
  "Welcome to constructor function..";  
}; let obj1 = new funOne(new  
  funtwo());  
console.log(obj1.test.test);  
o/p: Welcome to constructor function.
```

=>Nested Functions:

```
function function1()  
{  
  this.function2 = () =>  
{  
  console.log("Helloworld..");  
}}
```



```
    } }  
  
let myVar1 = new  
function1();  
myVar1.function2();  
o/p:  
  
Hello world..
```

Adding properties(Variables) to constructor functions dynamically

- Prototype is predefined keyword - used to add properties and functions to constructor function

Example:

```
function funcOne() {  
    } funcOne.prototype.wish =  
"Hello world.." let myObj = new  
funcOne();  
console.log(myObj.wish);
```

o/p: Hello World

Adding functions to constructor functions dynamically

```
function myFunction() {  
    } myFunction.prototype.fun_one =  
function() { return " From  
Function One";
```



```
}; myFunction.prototype.fun_two =  
function() {      return " From  
Function Two";  
}; myFunction.prototype.fun_three =  
function() {      return " From  
Function Three";  
}; let myObj = new myFunction();  
console.log(myObj.fun_one());  
console.log(myObj.fun_two());  
console.log(myObj.fun_three());
```

o/p:

From Function One

From Function Two

From Function Three

Combination of properties and functions to constructor functions dynamically

```
function myFunction() {  
  }  
myFunction.prototype.es6=  
"ES6";  
myFunction.prototype.fun_one =  
function() {      return ` From  
Function One ${this.es6} `; }; let  
myObj      =      new      myFunction();  
console.log(myObj.fun_one());
```



o/p:

From Function One ES6

- Inheritance is called as prototype chaining in JS
- Getting properties and functions from parent class is called as prototype chaining.
- Child and parent class relationship can be made using `Object.create(ParentClass)`

Example:

```
function mean() {  
  } mean.prototype.mean =  
  "MEAN Stack";
```

```
function mern() {  
  } mern.prototype = Object.create(mean.prototype);  
  //Mean is Parent class, Mern is c hild class  
  mern.prototype.mern = "MERN Stack"
```

```
function mevn() {  
  } mevn.prototype = Object.create(mern.prototype);  
  //Mern is Parent class, Mevn is c hild class  
  mevn.prototype.mevn = "MEVN Stack";  
  let mevnObj = new  
  mevn();  
  console.log(mevnObj.mean, mevnObj.mern, mevnObj.mevn);
```



o/p: MEAN Stack MERN Stack MEVN Stack

mean (parent)



mern (child to mean, parent to mevn)



mevn (child to mern and mean)

This process is called prototype chaining in JS.

Console.dir(ObjectName) : is used to see the internal structure of object.

Example:

Note: paste below code in a html file and then test.

```
<script>
    function myFunction() {
this.variable1 = "variable 1";
this.variable2 = "variable 2";
this.variable3 = "variable 3";
    }
myFunction.prototype.variable4 = "variable 4"
myFunction.prototype.variable5 = "variable 5"
console.dir(myFunction);
</script>
```

o/p:

f myFunction()



1. `arguments`: `null`
2. `caller`: `null`
3. `length`: `0`
4. `name`: `"myFunction"`
5. `prototype`:
 1. `variable4`: `"variable 4"`
 2. `variable5`: `"variable 5"`
 3. `constructor`: `f myFunction()`
 4. `__proto__`: `Object`
6. `__proto__`: `f ()`
7. `[[FunctionLocation]]`: [sampleHTML.html:12](#)
8. `[[Scopes]]`: `Scopes[1]`

Note: Arrow Functions cannot have Prototype.

➔ `Object.prototype` is the parent for all custom functions.

➔ All the functions available in `Object.prototype` are available in custom Fun's Eg: `bind()`, `toString()`, `apply()` etc etc etc...

For predefined functions:

➔ `Date.prototype` is Parent for Date

➔ `Time.prototype` is parent for Time



=>Optional Parameters in Functions

- While calling functions, few parameters are optional.
representation "?" Symbol
- Introduced in ES6
- Will work in TypeScript Environment
- File extension is .ts

Example:

```
function myFunction(arg1?:String, arg2?:string,arg3?:string){
  console.log(arg1,arg2,arg3);
} myFunction();

//undefined undefined undefined - Variable Hoisting
myFunction("ReactJs");

//ReactJs undefined undefined
myFunction("ReactJs","AngularJs","Typescript");

//ReactJs AngularJs Typescript
myFunction("ReactJs",undefined,"AngularJs");

//ReactJs undefined AngularJs
myFunction(undefined,undefined,undefined);

//undefined undefined undefined
myFunction(null,null,null);

//null null null
```

Combination of Regular parameter and optional parameter

Note: Optional parameter should be in last position in this combination.



Example:

```
function myFunction(arg1:string, arg2?:string, arg3?:string) {
  console.log(arg1, arg2, arg3);
}
```

```
// myFunction(); //Compilation Error - Expected 1 or 2 parameters
// but got zero
```

```
myFunction("ReactJs");
```

```
//ReactJs          undefined          undefined
```

```
myFunction("ReactJs", "AngularJs", "Typescript");
```

```
//ReactJs          AngularJs          Typescript
```

```
myFunction("ReactJs", undefined, "AngularJs");
```

```
//ReactJs          undefined          AngularJs
```

```
myFunction(undefined, undefined, undefined);
```

```
//undefined undefined undefined
```

```
myFunction(null, null, null);
```

```
//null null null
```

Combination of Regular Parameter + Optional Parameter + Default Parameter

```
Function myFunction(arg1:string,
  arg2?:string, arg3:string="Hello3") {
  console.log(arg1, arg2, arg3);
}
```

```
//myFunction();
```



```
//Compilation Error - Expected 1 or 2 parameters but got zero
myFunction("ReactJs");
//ReactJs                undefined                Hello3
myFunction("ReactJs","AngularJs","Typescript");
//ReactJs                AngularJs                Typescript
myFunction("ReactJs",undefined,"AngularJs");
//ReactJs                undefined                AngularJs
myFunction(undefined,undefined,undefined);
//undefined undefined Hello3
myFunction(null,null,null);
//null null null
```



Json(Java Script Object Notation) :

- JSON Stands for "Java Script Object Notation".
- JSON used to transfer the data over the "network".
- JSON also called as "JavaScript Objects".
- JSON is Network Friendly Format.
- Parsing(Reading) of JSON Eazy Compared to XML.
- JSON is lighth weight compared to XML.

Syntax

1) Objects ---- {}

2) Arrays ---- []

3) data ---- key & value pairs

key & value separated by using ":"

key & value pairs separated by using ","

Functions:

1) for....in

2) freeze()

3) seal()

4) defineProperty()

5) defineProperties()

6) concat()

7) fromEntries()

8) delete()



9) hasOwnProperty()

Examples:

=>

```
<script>

    let obj = {

        sub_one    : "ReactJS",

        sub_two    : "MicroServices",

        sub_three  : "Cloud"

    };

    console.log( obj.sub_one,

                obj.sub_two,

                obj.sub_three );

</script>
```

=>

```
<script>

    let obj = {"p_id":111};

    console.log(obj);    //{p_id: 111}

    Object.defineProperty(obj,{  "p_name":{

value:"p_one",
writable:false
},
```




```
"p_cost":{

value:10000,
writable:true
}

);

console.log(obj);    //{p_id: 111, p_name: "p_one", p_cost: 10000}

//p_id & p_cost -- we can update

//p_name -- we can't update

obj.p_id = "p101";

obj.p_cost=100000;

obj.p_name="product_one";

console.log(obj);

//{p_id: "p101", p_name: "p_one", p_cost: 100000}

</script>

=>

<script>

    let obj = {"p_id":111,

                "p_name":"p_one",

                "p_cost":10000};

    console.log(obj);

//{p_id: 111, p_name: "p_one", p_cost: 10000}
```



```
    console.log( Object.entries(obj) );

//[ ["p_id", 111], ["p_name", "p_one"], ["p_cost", 10000]]

//entries() function used to convert the Object to equivalent
Array

</script>

=>

//formEntries() function used to convert the array to
equivalent json object

<script>

    let arr = [{"eno","111"},

                ["ename","eone"],

                ["esal",10000]];

    console.log( Object.fromEntries(arr) );

    //{eno: "111", ename: "eone", esal: 10000}

</script>

=>

<script>

    let obj = {"p_id":111,"p_name":"p_one","p_cost":10000};

    console.log( obj.hasOwnProperty("p_id") );    //true

    console.log( obj.hasOwnProperty("pid") );    //false
```



```
</script>
```

```
=>
```

```
<script>
```

```
    let obj = {"p_id":111,"p_name":"p_one","p_cost":10000};
```

```
    console.log( Object.keys(obj) );
```

```
        //[ "p_id", "p_name", "p_cost" ]
```

```
    console.log( Object.values(obj) );
```

```
        //[111, "p_one", 10000]
```

```
</script>
```

```
=>
```

```
<script>
```

```
    let obj = {
```

```
        "login" : login
```

```
    };
```

```
    function login(){
```

```
        return "welcome to login module";
```

```
    };
```

```
    Console.log( obj.login() );    //welcome to login module
```

```
</script>
```

```
=>
```

```
<script>
```



```
let obj = {  
  "mean" : ()=>{  
    return "MEAN Stack Development Soon...!"  
  }  
};  
  
console.log( obj.mean() );    //MEAN Stack Development Soon...!  
</script>
```

=>

```
<script>  
  
  let arr = [  
    {"p_id":111,"p_name":"p_one","p_cost":10000},  
    {"p_id":222,"p_name":"p_two","p_cost":20000},  
    {"p_id":333,"p_name":"p_three","p_cost":30000},  
    {"p_id":444,"p_name":"p_four","p_cost":40000},  
    {"p_id":555,"p_name":"p_five","p_cost":50000}  
  ];  
  
  document.write(`  
  
    <table border="1"  
  
      align="center">  
  
    <thead>
```



```
        <tr>

            <th>SNO</th>

            <th>PID</th>

            <th>PNAME</th>

            <th>PCOST</th>

        </tr>

    </thead>

    <tbody>

`);

arr.forEach((element, index)=>{

    document.write(`<tr>

        <td>${index+1}</td>

        <td>${element.p_id}</td>

        <td>${element.p_name}</td>

        <td>${element.p_cost}</td>    </tr>

    </tbody>`); });

    document.write(`</table>`);

</script>
```

Rest Api (URLS) :

- read the data from following URL
URL : <https://restcountries.eu/rest/v2/all>



- above URL Technically called as Rest API Call.
- above Rest API Call gives th "JSON" as Response
- to know the structure of "JSON" we will visit following website

<http://jsonviewer.stack.hu/>

structure of above JSON

- initially we have JSON Array.
- we will iterate JSON array by using `forEach()` loop
- json array contains "250 JSON Objects".
- Each JSON Object contains following keys

@name

@capital

@region

@population

@nativeName

@flag

Example:

`<script>`

```
let arr ="copy paste above url data";
```

```
document.write(`<table border="1"
```



```
        align="center">

        <thead>

            <tr>

                <th>SNO</th>

                <th>NAME</th>

                <th>CAPITAL</th>

                <th>REGION</th>

                <th>POPULATION</th>

                <th>NATIVE NAME</th>

                <th>FLAG</th>

            </tr>

        </thead>

        <tbody>`;

        arr.forEach((element, index)=>{

            document.write(`

                <tr>

                    <td>${index+1}</td>

                    <td>${element.name}</td>

                    <td>${element.capital}</td>

                    <td>${element.region}</td>

                    <td>${element.population}</td>
```



```
        <td>${element.nativeName}</td>

        <td></td>

    </tr>

    `);

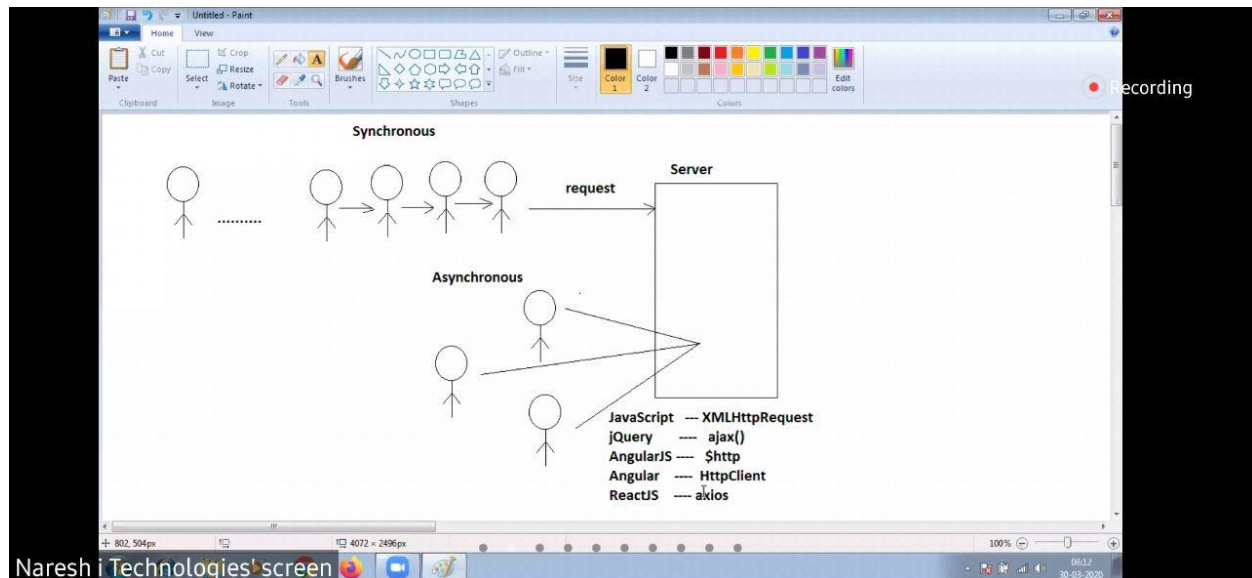
});

document.write(`</tbody></table>`);

</script>
```



Network Calls



- we can make network calls in 2 ways.

1) Synchronous

2) Asynchronous

- Executing Network calls one by one in sequential order called as Synchronous communication.
- Executing the network calls at a time (one network call may not effect to other network call) called as Asynchronous communication.
- now, we will discuss making asynchronous calls by using jQuery.
- we will include jQuery by using following CDN.

URL

:

<https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js>



Examples:

URL : `https://restcountries.eu/rest/v2/all`

Demo.html

```
<!DOCTYPE html>

<html>

  <head>

    <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.m
in.js"></script>

  </head>

  <body>

    <script>

      $.ajax({

        url:"https://restcountries.eu/rest/v2/all",

        method:"GET",

        success: (posRes) =>{

          console.log(posRes) ;

        },

        error: (errRes) =>{
```



```
        console.log(errRes) ;

    }

    });

</script>

</body>

</html>

=>

    URL : https://api.covid19india.org/data.json

<!DOCTYPE html>

<html>

    <head>

        <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.m
in.js"></script>

    </head>

    <body>

        <script>

            $.ajax({

                method: "GET" ,

                url: "https://api.covid19india.org/data.json" ,

                success: (posRes) =>{

                    console.log(posRes) ;
```



```
        },  
        error: (errRes) => {  
            console.log(errRes);  
        }  
    })  
    </script>  
    </body>  
</html>  
=>  
<!DOCTYPE html>  
<html>  
    <head>  
        <script  
src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.m  
in.js"></script>  
    </head>  
    <body>  
        <script>  
            $.ajax({  
                method: "GET",  
                url: "http://localhost:8080/product/getAll",  
                success : (posRes) => {
```



```
        console.log(posRes) ;

    },

    error : (errRes)=>{

        console.log(errRes) ;

    }

    })

</script>

</body>

</html>

=>

<!DOCTYPE html>

<html>

    <head>

        <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.m
in.js"></script>

    </head>

    <body>

        <script>

            $.ajax({

                method : "POST",

                url : "http://test-routes.herokuapp.com/test/uppercase",
```



```
data : {'message':'welcome to jquery'},  
succes : (posRes)=>{  
    console.log(posRes);  
},  
error : (errRes)=>{  
    console.log(errRes);  
}  
})  
</script>  
</body>  
</html>
```



Promises

- Promises establishes the communication between "producer" and "consumer".
- Promises are "special javascript object".
- we will create Promises by using "Promise" class.
- "Promise" is the predefined class in JavaScript.
 - "Promise" have two states.
 - 1) success (resolver)
 - 2) failure (reject)

Examples:

=>

<script>

```
let my_promise = new Promise((resolve,reject)=>{
    resolve("tomorrow i will discuss chaining in promises");
});

my_promise.then((posRes)=>{
    console.log(posRes);
}, (errRes)=>{
    console.log(errRes);
});

//tomorrow i will discuss chaining in promises
```



</script>

=>

<script>

```
let my_promise = new Promise((resolve, reject) => {  
    setTimeout(() => {  
        resolve("welcome to promises");  
    }, 5000);  
});  
  
my_promise.then((posRes) => {  
    console.log(posRes);  
}, (errRes) => {  
    console.log(errRes);  
}); //welcome to promises
```

</script>

=>

<script>

```
let my_promise = new Promise((resolve, reject) => {  
    reject("Failure");  
    resolve("Success");  
});
```




```
    });

    my_promise.then((posRes)=>{

        console.log(posRes);

    }, (errRes)=>{

        console.log(errRes);

    });
</script>

=>

<script>

    let my_promise = new Promise((resolve,reject)=>{

        setTimeout(()=>{

            resolve("Success");

        },3000);

        setTimeout(()=>{

            reject("Failure");

        },4000);

    });

    my_promise.then((posRes)=>{

        console.log(posRes);
```



```
    }, (errRes) => {  
        console.log(errRes);  
    });  
</script>
```

=>

```
let promise1 = new Promise((resolve, reject) => {  
    setTimeout(() => {  
        resolve("Hello_1");  
    }, 0);  
});  
  
let promise2 = new Promise((resolve, reject) => {  
    setTimeout(() => {  
        reject("Fail");  
    }, 5000);  
});  
  
let promise3 = new Promise((resolve, reject) => {  
    setTimeout(() => {  
        resolve("Hello_3");  
    }, 10000);  
});
```



```
});  
  
promise1.then((posRes)=>{  
    console.log(posRes);  
}, (errRes)=>{  
    console.log(errRes);  
});
```

```
promise2.then((posRes)=>{  
    console.log(posRes);  
}, (errRes)=>{  
    console.log(errRes);  
});
```

```
promise3.then((posRes)=>{  
    console.log(posRes);  
}, (errRes)=>{  
    console.log(errRes);  
});
```



```
Promise.all([promise1,promise2,promise3]).then((posRes)=>{  
    console.log(posRes[0],posRes[1],posRes[2]);  
},(errRes)=>{  
    console.log(errRes);  
});  
=>
```

- all() is the function present in Promise class, used to execute the multiple promises at a time
- we are getting the result based on max time of promises
- if any one promise fails, automatically application will execute only failure
- we will consume promises by using then() (as per old js)
- as per ES9, we will consume promises by using "async" & "await" keywords
- if we use "async" & "await" keywords automatically "application performance" will be increased
- if we use "async" & "await" keywords automatically "application readability" will be increased

Examples:

```
let promise1 = new Promise((resolve,reject)=>{  
    resolve("Success");  
});  
  
async function myFun(){  
    let res = await promise1;
```



```
        console.log(res);
    };

    myFun();    //Success

=>

let promise1 = new Promise((resolve, reject) => {
    setTimeout(() => {
        resolve("Hello_1");
    }, 5000);
});

let promise2 = new Promise((resolve, reject) => {
    setTimeout(() => {
        resolve("Hello_2");
    }, 10000);
});

async function myFun() {
    let res1 = await promise1;
    let res2 = await promise2;
    console.log(res1);
    console.log(res2);
};
```



```
myFun();
```

```
=>
```

```
function fun_one(num) {  
    return new Promise((resolve, reject) => {  
        resolve(num + 5);  
    });  
};
```

```
function fun_two(num) {  
    return new Promise((resolve, reject) => {  
        resolve(num - 3);  
    });  
};
```

```
async function consume() {  
    let res1 = await fun_one(5);  
    let res2 = await fun_two(res1);  
    console.log(res2);  
};  
  
consume();  
  
=>
```



AJAX Calls

Promises

async & await

```
<!DOCTYPE html>

<html>

  <head>

    <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.m
in.js"></script>

  </head>

  <body>

    <script>

      function getCountries() {

        return new Promise((resolve, reject) => {

          $.ajax({

            method: "GET",

            url: "https://www.w3schools.com/angular/customers.php",

            success: (posRes) => {

              resolve(posRes);

            },

            error: (errRes) => {
```



```
        reject(errRes);
    }
    })
    });
};

async function consume(){
    let res = await getCountries();
    console.log(res);
};

consume();
</script>
</body>
</html>
```



Array Manipulations

<i>findIndex()</i> - immutable	Know the index of element in array - <code>myArray.findIndex((element) => { return element === 30; })</code>
<i>Find()</i> - immutable	To know whether element is present in array or not. If element is present - will return same element. Else - returns undefined. - <code>myArray.find((element) => { return element === 30; }});</code>
<i>Includes()</i> - immutable	similar to find - but varies at return value If element present - return true. Else - returns false. → <code>myArray.includes(30);</code>
<i>CopyWithin()</i> - mutable	Shift the elements in an array. <code>myArray.copyWithin(3)</code>
<i>indexOf()</i> - immutable	Know the index of first element present in array, will return first matched element index → if found will return index Else will return -1 → <code>myArray.indexOf(10)</code>
<i>Sort()</i> - mutable	Sort array based on ASC order or DESC order. → <code>myArray.sort((arg1, arg2) => { return arg1 - arg2; }})</code>
<i>Reverse()</i> - mutable	To reverse the array elements <code>myArray.reverse()</code>
<i>toString()</i> - immutable	Convert array to string disAdv : we will be getting additional commas → <code>myArray.toString()</code>
<i>Fill()</i> - mutable	To replace the existing array elements with new elements. → <code>myArray.fill(100)</code>



<i>Flat()</i> - mutable	<i>Used to convert multi-dimensional array to single dimensional array. → <code>multiDimArray.flat(1)</code></i>
<i>FlatMap()</i> - immutable	<i>Map two arrays in to one array → <code>myArray1.flatMap((element, index) => { return [element, myArray2[index]]; }));</code></i>

1) `map()`

- it is used to manipulate the each and every array element

Examples:

```
console.log(  
  [1,2,3,4,5].map((element,index)=>{  
    return element*10;  
  })  
);    //[ 10, 20, 30, 40, 50 ]  
  
console.log(  
  [100,1000,200,2000,300,3000].map((element,index)=>{  
    // if(index === 0){  
    //   return "$"+element;  
    // }else if(index === 1){  
    //   return "INR"+element;  
    // }else if(index === 2){
```



```
//      return "€"+element;

// }else if(index === 3){

//      return "£"+element;

// }else{

//      return "¥"+element;

// }

switch(index){

    case 0:

        return "$"+element;

    case 1:

        return "INR"+element;

    case 2:

        return "€"+element;

    case 3:

        return "£"+element;

    default:

        return "¥"+element;

}

});
```

2) filter()

- it is used to create the new array based on conditions



```
console.log(
    [1000,2000,3000,4000,5000].filter((element,index)=>{
        return element>=3000;
    })
);    //[ 3000, 4000, 5000 ]

console.log(
    [1,2,3,4,5].map((element,index)=>{
        return element*10000;
    }).filter((element,index)=>{
        return element>=40000;
    })
);    //[ 40000, 50000 ]
```

3) reduce()

- it is used to find the sum of array elements

```
console.log(
    [1,2,3,4,5].reduce((firstValue,nextValue)=>{
        return firstValue+nextValue;
    })
);    //15

console.log(
```



```
[100,200,300,400,500].map((element,index)=>{  
    return element-50;  
}).filter((element,index)=>{  
    return element<=150;  
}).reduce((firstValue,nextValue)=>{  
    return firstValue+nextValue;  
})  
);
```

4) reduceRight()

- it is used to find the sum of elements from "right to left" in array

```
console.log(
```

```
["ReactJS","To","Welcome"].reduceRight((firstValue,nextValue)=>{  
    return firstValue+" "+nextValue;  
})  
); //Welcome To ReactJS
```

5) push()



```
let arr = [20,30,40];  
  
console.log(arr);      //[ 20, 30, 40 ]  
  
arr.push(50);  
  
console.log(arr);      //[ 20, 30, 40, 50 ]
```

6) unshift()

```
arr.unshift(10);  
  
console.log(arr);      //[ 10, 20, 30, 40, 50 ]
```

7) pop()

```
arr.pop();  
  
console.log(arr);      //[ 10, 20, 30, 40 ]
```

8) shift()

```
arr.shift();  
  
console.log(arr);      //[ 20, 30, 40 ]
```

9) splice()

```
let arr = [10,20,30,40,50,60,70,80,90,100];  
  
arr.splice(4,2);  
  
console.log(arr);      //[10,20,30,40,70,80,90,100]  
  
//arr.splice(4,1);  
  
//console.log(arr);      //70 deleted successfully  
  
arr.splice(-4,1);      //70 deleted successfully  
  
console.log(arr);      [10,20,30,40,80,90,100]
```



```
arr.splice(-6,2);  
  
console.log(arr);      //[ 10, 40, 80, 90, 100 ]  
  
arr.splice(1,0,20,30);  
  
console.log(arr);      //[10, 20, 30, 40, 80, 90, 100]  
  
arr.splice(4,0,50,60,70);  
  
console.log(arr);
```

10) findIndex()

- it is used to find the index of particular element in array

=>

```
let arr = [10,20,30,40,50];  
  
console.log(  
    arr.findIndex((element,index)=>{  
        return element === 30;  
    })  
);    //2  
  
=>  
  
let arr1 = [10,100,1000,10000,20,200,2000,20000];  
  
arr1.splice(arr1.findIndex((element,index)=>{  
    return element === 20;  
})
```



```
   )),1);  
  
console.log(arr1);  
  
=>  
  
let arr2 = [{"price":10000},  
            {"price":20000},  
            {"price":30000},  
            {"price":40000},  
            {"price":50000}];  
  
arr2.splice(arr2.findIndex((element,index)=>{  
    return element.price === 30000;  
}),1);  
  
console.log(arr2);
```

11) indexOf()

- it won't create indexes to repeated elements

```
let arr = [10,20,30,40,10,50,20,60,40];  
  
arr.forEach((element,index)=>{  
    console.log( arr.indexOf(element) );  
});
```

```
let arr = [10,20,30,10,20,30];
```




```
arr.forEach((element,index)=>{  
    console.log( arr.indexOf(element) );  
});  
  
let arr = [1,2,1,3,1,4,5,4,2];  
  
console.log(  
    arr.filter((element,index)=>{  
        return arr.indexOf(element) === index;  
    })  
);    //[ 1, 2, 3, 4, 5 ]
```

12) copyWithin()

- Shift the elements in an array.

```
let arr1 = [10,20,30,40,50,60,70,80,90,100];  
arr1.copyWithin(2);  
console.log(arr1);  
  
let arr2 = [10,20,30,40,50,60,70,80,90,100];  
arr2.copyWithin(5);  
console.log(arr2);  
  
let arr3 = [10,20,30,40,50,60,70,80,90,100];  
arr3.copyWithin(2,5);
```



```
console.log(arr3);

let arr4 = [10,20,30,40,50,60,70,80,90,100];

arr4.copyWithin(0,9);

console.log(arr4);

let arr5 = [10,20,30,40,50,60,70,80,90,100];

arr5.copyWithin(1,3,6);

console.log(arr5);
```

13) slice()

```
let arr = [10,20,30,40,50,60,70,80,90,100];

console.log(arr.slice(4,6));           //[ 50, 60 ]
console.log(arr.slice(6,9));           //[ 70, 80, 90 ]
console.log(arr.slice(7));             //[ 80, 90, 100 ]
console.log(arr.slice(2,-1));          //[ 30, 40, 50, 60,70, 80, 90]
console.log(arr.slice(2,-8));          //[ ]
console.log(arr.slice(-2));            //[ 90, 100 ]
console.log(arr.slice(-5,-2));         //[ 60, 70, 80 ]
```

14) length

```
let arr = [10,20,30,40,50,60,70,80,90,100];

console.log(arr.length);               //10
console.log( arr[0] );                 //10
console.log( arr[9] );                 //100
```



```
console.log( arr[10] );           //undefined
console.log( arr[-1] );           //undefined
arr.length = 5;
console.log(arr.length);          //5
console.log(arr[0]);              //10
console.log(arr[5]);              //undefined
console.log(arr[arr.length-1]);   //50

let arr = [10,20,30,40,50];
delete arr[2];
console.log(arr.length);          //5
console.log(arr);                 //[ 10, 20, <1 empty item>, 40, 50
]
```

15) every()

```
let arr = [10,20,30,40,50];
console.log(
    arr.every((element,index)=>{
        return element>=10;
    })
);
```



```
    })  
);    //true  
  
16) some()  
  
let arr = [10,20,30,40,50];  
  
console.log(  
    arr.some((element,index)=>{  
        return element>50;  
    })  
);    //false
```

```
17) find()  
  
let arr = [10,20,30,40,50];  
  
console.log(  
    arr.find((element,index)=>{  
        return element === 3;  
    })  
);
```

```
18) reverse()  
  
console.log(  
    [10,20,30].reverse()  
);
```



```
console.log(  
    ["Angular", "ReactJS", "NodeJS"].reverse()  
);
```

```
console.log(  
    Array.from("hello").reverse().join("")  
);    //olleh
```

```
// ["Angular", "NodeJS", "MongoDB"]
```

```
// ["ralugnA", "SJedoN", "BDognoM"]
```

```
// ["BDognoM", "SJedoN", "ralugnA"]
```

19) toString()

```
console.log(  
    ['h', 'e', 'l', 'l', 'o'].toString()  
    ['h', 'e', 'l', 'l', 'o'].join("")  
);    //hello
```

20) replace()

```
console.log(  
    ["w", "e", "l", "c", "o", "m", "e"].toString().replace(/,/g, "")  
);    //welcome
```

21) repeat()

```
console.log(  

```



```
        "hello".repeat(5)

    );

22) concat()

let arr1 = ["Angular"];
let arr2 = ["NodeJS"];
let arr3 = ["MongoDB"];

console.log(

    arr1.concat(arr2,arr3)

);    //[ 'Angular', 'NodeJS', 'MongoDB' ]

console.log(

    [...arr1,...arr2,...arr3]

);    //[ 'Angular', 'NodeJS', 'MongoDB' ]

23) includes()

console.log(

    ["Angular","NodeJS","MongoDB"].includes("Node")

);

//true

//false

24) entries()

let obj = {
```



```
    "e_id":111,

    "e_name":"e_one",

    "e_sal":10000

};

let arr = Object.entries(obj);

console.log(arr);    //[ [ 'e_id', 111 ], [ 'e_name', 'e_one' ],
[ 'e_sal', 10000 ] ]

console.log( Object.fromEntries(arr) );    //{ e_id: 111, e_name:
'e_one', e_sal: 10000 }
```

25) assign()

```
let obj1 = {"e_id":111};

let obj2 = {"e_name":"e_one"};

let obj3 = {"e_sal":10000};

console.log( Object.assign(obj1,obj2,obj3) );

console.log( {...obj1,...obj2,...obj3} );
```

26) fill()

```
let arr = [1,2,3,4,5];

console.log( arr.fill(100) );    //[ 100, 100, 100, 100, 100 ]

console.log( arr.fill(200,2) );    //[ 100, 100, 200, 200, 200 ]

console.log( arr.fill(300,1,3) );    //[ 100, 300, 300, 200, 200 ]
```



27) flat()

```
let arr = [1,[2],[3],4];

console.log( arr.flat(1).reduce((firstValue,secondValue)=>{

    return firstValue+secondValue;

}) );      //10
```

28) reduce()

```
let arr = [1,[2,[3]],[4,[5]]];

console.log(

    arr.flat(2).reduce((firstValue,secondValue)=>{

        return firstValue+secondValue;

    })

); //15
```

29) flat()

```
let arr = [1,[[[[[[[[[[[[[2]]]]]]]]]]]]];

console.log(

    arr.flat(Infinity)

);    //[ 1, 2 ]
```

30) flatMap()

```
let arr1 = [1,2,3];

let arr2 = ["one","two","three"];
```




```
console.log(

  arr1.map((element,index)=>{

    return [element,arr2[index]];

  })

); // [ [ 1, 'one' ], [ 2, 'two' ], [ 3, 'three' ] ]
```

```
console.log(

  arr1.flatMap((element,index)=>{

    return [element,arr2[index]];

  })

); // [ 1, 'one', 2, 'two', 3, 'three' ]
```

```
console.log(

  arr1.map((element,index)=>{

    return [element,arr2[index]];

  }).flat(1)

); // [ 1, 'one', 2, 'two', 3, 'three' ]
```

```
31) substr()
```

```
console.log(

  "welcome to reactjs".substr(0,7)

);
```



```
console.log(
    "welcome to reactjs".substr(8,2)
);

console.log(
    "welcome to reactjs".substr(11)
);

32) sort()

console.log(
    [10,50,20,40,30].sort((num1,num2)=>{
        return num1-num2;
    })
);    //[ 10, 20, 30, 40, 50 ]
```

```
console.log(
    [10,50,20,40,30].sort((num1,num2)=>{
        return num1-num2;
    })[1]
);    //2nd min element : 20
```

```
console.log(
    [10,50,20,40,30].sort((num1,num2)=>{
```



```
        return num2-num1;

    })

); // [ 50, 40, 30, 20, 10 ]
```

```
console.log(

    [10,50,20,40,30].sort((num1,num2)=>{

        return num2-num1;

    })[1]

); // 2nd max element : 40
```

33) lastIndexOf()

```
let arr = [10,20,30,40,50,10,20,40,10];

console.log( arr.lastIndexOf(10) );    //8
console.log( arr.lastIndexOf(20) );    //6
console.log( arr.lastIndexOf(30) );    //2

console.log( arr.lastIndexOf(10,5) );    //5
console.log( arr.lastIndexOf(30,1) );    //-1
```

34) set()

```
let map = new Map();

map.set("key1",10);

map.set("key2",20);
```



```
map.set("key3",30);  
  
for(let [k,v] of map){  
    console.log(k,v);  
}
```



Callbacks

passing "one function defination" to "another function as an argument" called as callback

Examples:

=>

```
function fun_one(arg1){  
    console.log( arg1() );  
};  
  
function fun_two(){  
    return "hello";  
};  
  
fun_one(fun_two);
```

=>

```
function fun_one(arg1){  
    console.log( arg1() );  
};  
  
fun_one( function fun_two(){  
    return "hello";  
} );    //hello
```



=>

```
function mern(arg1,arg2,arg3){  
    console.log( arg1(), arg2(), arg3() );  
};  
mern( ()=>{  
    return "ReactJS";  
}, ()=>{  
    return "NodeJS";  
}, ()=>{  
    return "MongoDB";  
});    //ReactJS NodeJS MongoDB
```

=>

```
function add(num,callback){  
    return callback( num+5, false );  
};  
function sub(num,callback){  
    return callback(num-3,false);  
};  
function mul(num,callback){  
    return callback(num*2,false);  
};
```



```
function div(num,callback){  
    return callback(num/2,false);  
}  
  
add(5,(addRes,error)=>{  
    if(!error){  
        sub(addRes,(subRes,error)=>{  
            if(!error){  
                mul(subRes,(mulRes,error)=>{  
                    if(!error){  
                        div(mulRes,(divRes,error)=>{  
                            if(!error){  
                                console.log(divRes);  
                            }  
                        });  
                    }  
                });  
            }  
        });  
    }  
});  
});
```



=>

```
function add(num) {  
    return new Promise((resolve, reject) => {  
        resolve(num + 5);  
    });  
};  
  
function sub(num) {  
    return new Promise((resolve, reject) => {  
        resolve(num - 3);  
    });  
};  
  
function mul(num) {  
    return new Promise((resolve, reject) => {  
        resolve(num * 2);  
    });  
};  
  
function div(num) {  
    return new Promise((resolve, reject) => {  
        resolve(num / 2);  
    });  
};
```




```
async function cal(){  
  
    let addRes = await add(5);  
  
    let subRes = await sub(addRes);  
  
    let mulRes = await mul(subRes);  
  
    let divRes = await div(mulRes);  
  
    console.log(divRes);  
  
};  
  
cal();
```



data structures

JavaScript supports following data structures

- 1) Map
- 2) WeakMap
- 3) Set
- 4) WeakSet

Examples:

=>

<script>

```
let map = new Map();

console.dir(map); /*

  clear()

  constructor()

  delete()

  entries() / forEach() / for...of()

  get()

  has()

  keys()

  set()

  size()

  values() */ </script>
```



=>

<script>

```
let map = new Map();

map.set("key1", "ReactJS")

    .set("key2", "Angular")

    .set("key3", "NodeJS")

    .set("key4", "MongoDB");

console.log(map);

console.log( map.keys() );

console.log( map.values() );

console.log( map.has("key1") );

console.log( map.get("key1") );

map.delete("key4");

console.log( map );

for(let [k,v] of map){

    console.log(k,v);

}

console.log( map.size );    //3

map.clear();

console.log( map );
```

</script>



=>

```
<script>
```

```
    let map = new Map();

    map.set("key1", "Hello_1");

    map.set("key1", "Hello_2");

    map.set({}, "Hello_3");

    map.set({}, "Hello_4");

    console.log( map );

    for(let [k,v] of map){

        console.log(k,v);

    };

```

```
</script>
```

WeakMap

- WeakMap allows only keys are "JSON Objects"

Examples:

```
<script>
```

```
    let map = new WeakMap();

    map.set({}, "Hello_1");

    map.set("key1", "Hello_2");

    console.log(map);

```

```
</script>
```



=>

```
<script>
```

```
    let set = new Set();
```

```
    set.add(10);
```

```
    set.add(20);
```

```
    set.add(30);
```

```
    set.add(10);
```

```
    set.add(40);
```

```
    set.add(20);
```

```
    console.log(set);
```

```
</script>
```

=>

```
<script>
```

```
    let arr = [10,20,10,20,30];
```

```
    console.log(
```

```
        [...new Set(arr)]
```

```
    );
```

```
</script>
```

=><script>

```
    let set = new Set();
```

```
    console.dir(set); </script>
```



=>

<script>

```
let set = new WeakSet();  
  
let obj1 = {val:10};  
  
let obj2 = {val:20};  
  
set.add(obj1)  
  
    .add(obj2);  
  
console.log(set);  
  
console.log( set.has(obj1) );  
  
set.delete(obj2);  
  
console.log(set);
```

</script>



Closure:

→ Internal function using properties / variable defined in outer function.

→ used for security purpose - so that

→ Closure is useful in hiding implementation detail in JavaScript. In other words, it can be useful to create private variables or functions.

→ The following example shows how to create private functions & variable.

Example:

```
let myFunction = function() {    let i = 10,        j = 20;
return (() => {                console.log(i, j); // internal arrow
function using outer function variables.
    }) }
myFunction()(); //
10,20
```

Global Polluting Issue: -- important topic for interviews..!

```
for (var i = 0; i <
5; i++) {
setTimeout(() => {
console.log(i);
    }, 5000);
} 5 5 5 5 5
```



→ After 5 seconds 5 will be displayed 5 times. Because within 5 seconds the loop finishes the execution and by then the value of I will be 5. Hence that latest value of I will be considered and printed.

→ This Global polluting issue can be solved in 2 ways

- Using let key word (< ES6)
- Using IIFE (>=ES9)

Using Let Keyword: `for`

```
(let i = 0; i < 5; i++) {
  setTimeout(() => {
    console.log(i);
  }, 5000);
} o/p: 0 1
2 3 4
```

→ Using IIFE:

```
for (var i = 0; i < 5; i++) {
  ((i) => {
    setTimeout(() => {
      console.log(i);
    }, 5000)
  })(i);
} //0 1 2
3 4
```

IIFE:

⇒ Immediate Invocable Function Expression, introduced in ES9



⇒ They are self invocable functions.

⇒ Syntax: `((() => {}))();`

Example:

```
((() => {  
  console.log("Welcome to  
  IIFE");  
})());  
o/p:  Welcome to IIFE
```

Passing parameters to IIFE:

```
((arg1, arg2, arg3) => {  
  console.log(arg1, arg2, arg3);  
})("Angular", "ReactJs",  
  "VueJs");  
// Angular ReactJs  
VueJs
```

Combination of IIFE and reverse():

```
((arg1) => {  
  
  console.log(Array.from(arg1).reverse().toString().replace(/,/g,  
  ""));  
})("Angular");  
//O/P: ralugnA
```

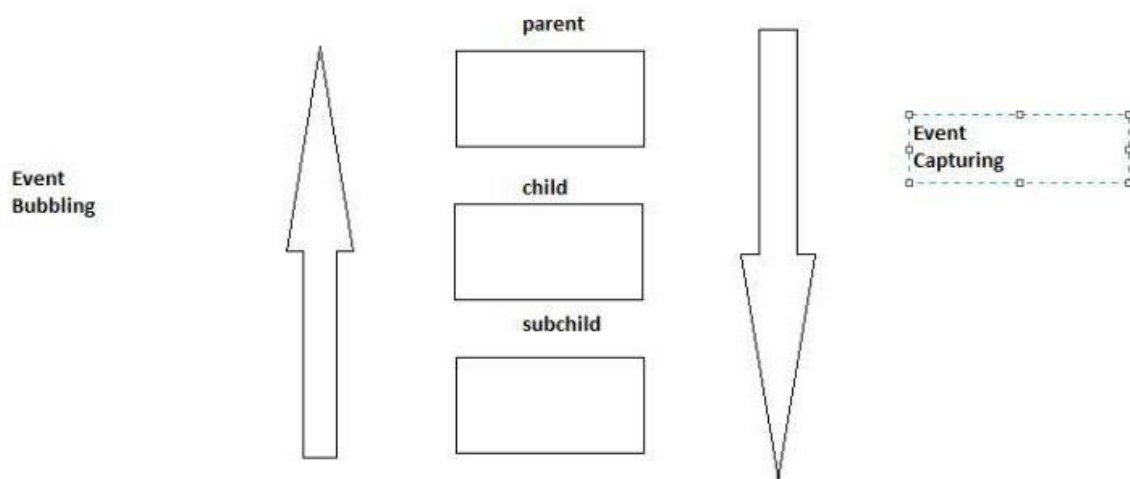
Combination of ASYNC and AWAIT in IIFE

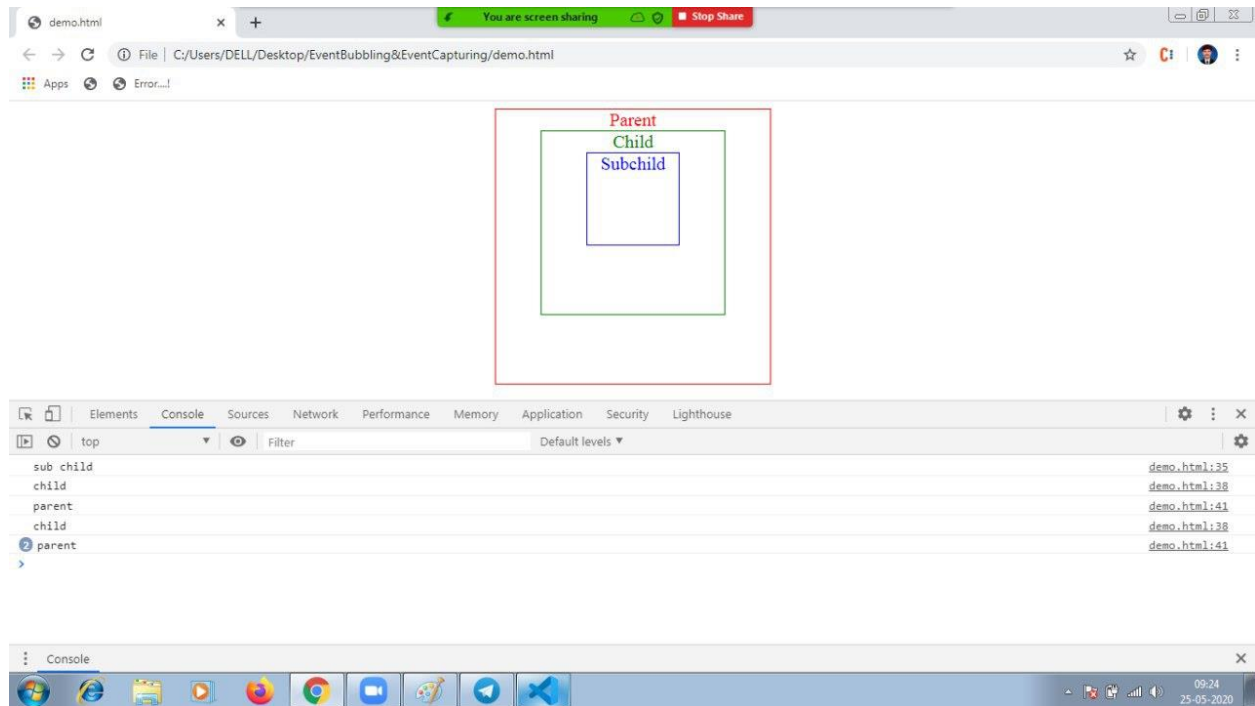


```
let answer = (async() => {    let res = await
Promise.allSettled([promise1, promise2, promise3]);
console.log({...res[0].value, ...res[1].value,
...res[2].value });
})(); o/p:

//{ sub_one: 'Angular', sub_two: 'ReactJs', sub_three:
```

Event Bubbling&Event Capturing





Example:

```
<html>
```

```
  <head>
```

```
    <script  
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.m  
in.js"></script>
```

```
  </head>
```

```
  <body>
```

```
    <div id="parent"  style="width: 300px;  
                        height: 300px;
```



```
border: 1px solid red;  
  
margin: auto;  
  
text-align: center;  
  
font-size: 20px;  
  
color: red;">
```

Parent

```
<div id="child" style="width: 200px;  
  
    height: 200px;  
  
    border: 1px solid green;  
  
    margin: auto;  
  
    text-align: center;  
  
    font-size: 20px;  
  
    color: green;">
```

Child

```
<div id="subchild" style="width: 100px;  
  
    height: 100px;  
  
    border: 1px solid blue;  
  
    margin: auto;  
  
    text-align: center;  
  
    font-size: 20px;  
  
    color: blue;">
```



```
        Subchild
    </div>
</div>
<script>
    /*
    Event Bubbling

    $("#subchild").click(function() {
        console.log("sub child");
    });

    $("#child").click(function(event) {
        event.stopPropagation();
        console.log("child");
    });

    $("#parent").click(function() {
        console.log("parent");
    });
    */

    //Event Capturing

    let parent = document.querySelector("#parent");
    let child = document.querySelector("#child");
```



```
    let subchild = document.querySelector("#subchild");

    parent.addEventListener("click",function(){

        console.log("parent");

    },true);

    child.addEventListener("click",function(){

        console.log("child");

    },true);

    subchild.addEventListener("click",function(){

        console.log("subchild");

    },true);

</script>

</body> </html>
```

Generators

Examples:

=>

```
function *fun_one(){

    yield 10;

    yield 20;

    yield 30;

    yield 40;
```



```
    yield 50;

};

let cursor = fun_one();

console.log( cursor.next() );

console.log( cursor.next() );

console.log( cursor.next() );

console.log( cursor.next() );

console.log( cursor.next() );

console.log( cursor.next() );
```

=>

```
function *fun_one(){

    yield "Angular";

    yield *fun_two();

    yield "ReactJS";

};
```

```
function *fun_two(){

    yield "NodeJS";

};
```



```
};
```

```
let cursor = fun_one();  
console.log( cursor.next() );  
console.log( cursor.next() );  
console.log( cursor.next() );  
console.log( cursor.next() );  
console.log( cursor.next() );
```

=>

```
function *fun_one(){  
    yield 10;  
    yield 20;  
    return 30;  
    yield 40;  
};  
  
let cursor = fun_one();  
console.log( cursor.next() );  
console.log( cursor.next() );  
console.log( cursor.next() );  
console.log( cursor.next() );  
console.log( cursor.next() );
```

=>

```
let obj = { num1:10 };
```




```
function fun_one(num2,num3,num4) {  
    console.log( this.num1 + num2+ num3+ num4 );  
};  
  
fun_one.call(obj,20,30,40);    //obj  
  
=>  
  
let obj = {  
    arg1:1,  
    arg2:2  
};  
  
function fun_one(arg1,arg2,arg3) {  
    console.log( this.arg1+ this.arg2 + arg1 + arg2 + arg3 );  
};  
  
fun_one.call( obj, 3, 4, 5 );    //15  
  
fun_one.apply( obj, [30,40,50] ); //123  
  
let newFun = fun_one.bind(obj);  
  
newFun(100,200,300);    //603
```



